

OOPJ CH 1:

1. Java ના વિવિધ features સમજાવો

Java એક લોકપ્રિય, શક્તિશાળી અને object-oriented programming language છે. તેના મુખ્ય features નીચે મુજબ છે:

1) Simple (સરળ)

Java નું syntax C/C++ જેવી ભાષાઓથી થોડું મળતું આવે છે, પણ તે વધુ સરળ છે.

Java માં pointer, operator overloading, multiple inheritance by classes જેવી જટિલ બાબતો નથી, તેથી શીખવા અને લખવા માટે સરળ છે.

2) Object-Oriented

Java સંપૂર્ણ રીતે object-oriented language છે.

એમાં દરેક વસ્તુ class અને object ના રૂપમાં હોય છે.

OOP ના મુખ્ય concepts:

- Class
- Object
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction

3) Platform Independent

Java નો code એક વાર લખ્યા પછી કોઈપણ operating system પર ચલાવી શકાય છે.

કારણ:

Java code પહેલા bytecode માં compile થાય છે, અને પછી JVM (Java Virtual Machine) તેને ચલાવે છે.

આથી Java નું famous statement છે:

Write Once, Run Anywhere

4) Secure

Java security ને ધ્યાનમાં રાખીને design કરવામાં આવી છે.
એમાં:

- pointer નથી
- bytecode verifier છે
- sandbox model છે

આથી Java programs વધારે secure માનવામાં આવે છે.

5) Robust

Robust એટલે મજબૂત અને reliable.
Java માં:

- strong memory management
- exception handling
- garbage collection

આ બધાના કારણે Java programs વધુ stable રહે છે.

6) Multithreaded

Java એક જ સમયે ઘણા tasks ચલાવી શકે છે.
એને multithreading કહે છે.

ઉદાહરણ:

- music play થતું હોય અને સાથે download પણ ચાલતું હોય
- game માં એક સાથે અનેક operations

7) Portable

Java bytecode કોઈપણ machine પર ચલાવી શકાય છે, જો ત્યાં JVM હોય.
એથી Java portable છે.

8) High Performance

JIT (Just-In-Time) compiler ના કારણે Java ની performance સારી થાય છે.

9) Distributed

Java internet અને network applications માટે useful છે.
તે distributed systems બનાવવા માટે મદદ કરે છે.

10) Dynamic

Java runtime દરમિયાન classes load કરી શકે છે.
એથી Java dynamic behavior support કરે છે.

2. Java માં main() method સમજાવો

main() method એ Java program નું entry point છે.
Program execution અહીંથી શરૂ થાય છે.

Syntax

```
public static void main(String[] args)
{
    // statements
}
```

દરેક keyword નું અર્થ

public

આ method JVM દ્વારા anywhere થી access કરી શકાય છે.

static

Object બનાવ્યા વગર method call થઈ શકે છે.

void

આ method કોઈ value return કરતી નથી.

main

આ special method નું નામ છે. Program execution માટે JVM આ method શોધે છે.

String[] args

Command line arguments store કરવા માટેનો array છે.

Example

```
class Demo
{
    public static void main(String[] args)
    {
        System.out.println("Hello Java");
    }
}
```

Output

Hello Java

3. Java ની primitive data types સમજવો

Java માં કુલ 8 primitive data types હોય છે.

Data Type	Size	Use
byte	1 byte	નાના integer numbers
short	2 bytes	મધ્યમ integer numbers
int	4 bytes	સામાન્ય integer numbers
long	8 bytes	મોટા integer numbers
float	4 bytes	decimal values
double	8 bytes	વધુ precision વાળા decimal values
char	2 bytes	single character
boolean	1 bit	true/false values

1) byte

- Size: 1 byte
- Range: -128 to 127

Example:

```
byte b = 10;
```

2) short

- Size: 2 bytes

Example:

```
short s = 200;
```

3) int

- Size: 4 bytes
- સૌથી વધુ ઉપયોગ થતો integer type

Example:

```
int n = 10000;
```

4) long

- Size: 8 bytes
- મોટા numbers માટે

Example:

```
long l = 1234567890L;
```

5) float

- Size: 4 bytes
- decimal values માટે
- અંતે f લગાવવું પડે

Example:

```
float f = 5.75f;
```

6) double

- Size: 8 bytes
- float કરતાં વધુ precise

Example:

```
double d = 99.12345;
```

7) char

- Size: 2 bytes
- એક character store કરે છે

Example:

```
char ch = 'A';
```

8) boolean

- value: true અથવા false

Example:

```
boolean result = true;
```

4. Java માં operators ના વિવિધ types સમજાવો

Operator એ symbol છે જે data પર operation કરે છે.

1) Arithmetic Operators

ગણિતીય operations માટે.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

```
int a = 10, b = 3;  
System.out.println(a + b); // 13  
System.out.println(a % b); // 1
```

2) Relational Operators

બે values ની comparison માટે.

Operator	Meaning
==	equal to
!=	not equal to
>	greater than

Operator	Meaning
<	less than
>=	greater than or equal to
<=	less than or equal to

Example:

```
int a = 10, b = 20;
System.out.println(a < b);    // true
System.out.println(a == b);  // false
```

3) Logical Operators

conditions ને combine કરવા માટે.

Operator Meaning

&& AND

! NOT

Example:

```
int a = 10, b = 20;
System.out.println(a > 5 && b > 10); // true
```

4) Assignment Operators

value assign કરવા માટે.

Operator Meaning

= assign

+= add and assign

-= subtract and assign

*= multiply and assign

/= divide and assign

%= modulus and assign

Example:

```
int a = 10;
a += 5;
System.out.println(a); // 15
```

5) Unary Operators

એક જ operand પર કામ કરે છે.

Operator	Meaning
----------	---------

+	positive
-	negative
++	increment
--	decrement
!	logical NOT

Example:

```
int a = 5;  
System.out.println(++a); // 6
```

6) Bitwise Operators

bits પર operation કરે છે.

Operator	Meaning
----------	---------

&	bitwise AND
^	bitwise XOR
~	bitwise complement
<<	left shift
>>	right shift

Example:

```
int a = 5, b = 3;  
System.out.println(a & b);
```

7) Ternary Operator

શરત પર based short form if-else.

Syntax:

```
condition ? expression1 : expression2
```

Example:

```
int a = 10, b = 20;  
String result = (a > b) ? "A is bigger" : "B is bigger";  
System.out.println(result);
```

8) Special Operators

- instanceof — object કોઈ class નો છે કે નહીં તે check કરે છે
 - . — member access માટે
 - [] — array access માટે
-

5. Type conversion અને casting શું છે? Example સાથે સમજાવો

Type Conversion

એક data type ને automatically બીજા larger data type માં બદલવું.

આને **widening conversion** પણ કહે છે.

Example:

```
int a = 10;  
double d = a;  
System.out.println(d);
```

અહીં int થી double automatic conversion થાય છે.

Type Casting

જ્યારે programmer manually એક data type ને બીજા data type માં બદલે, તેને type casting કહે છે.

આને **narrowing conversion** પણ કહે છે.

Syntax:

```
targetType variable = (targetType) value;
```

Example:

```
double d = 10.75;  
int a = (int) d;  
System.out.println(a);
```

Output:

10

અહીં decimal ભાગ remove થઈ જાય છે.

Difference

Type Conversion	Type Casting
automatic	manual
smaller to larger	larger to smaller
safe	data loss શક્ય

6. switch statement example સાથે સમજાવો

switch statement ઘણા options માંથી એક પસંદ કરવા માટે વપરાય છે.

Syntax

```
switch(expression)  
{  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    default:  
        // statements  
}
```

Example

```
class SwitchDemo  
{  
    public static void main(String[] args)  
    {  
        int day = 3;
```

```
switch(day)
{
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;
    default:
        System.out.println("Invalid day");
}
}
```

Output

Wednesday

break નું કામ

break statement case પૂરો થતાં જ switch blockમાંથી બહાર કાઢે છે.
જો break ન લખીએ તો next cases પણ execute થઈ શકે છે.

7. Loop ની types list કરો અને વિગતે સમજાવો

Loop નો ઉપયોગ code ને વારંવાર ચલાવવા માટે થાય છે.

Java માં મુખ્ય loops:

1. for loop
2. while loop
3. do-while loop
4. enhanced for loop

1) for loop

જ્યારે iterations count પહેલેથી ખબર હોય ત્યારે.

Syntax

```
for(initialization; condition; update)
{
    // statements
}
```

Example

```
for(int i = 1; i <= 5; i++)
{
    System.out.println(i);
}
```

Output

```
1
2
3
4
5
```

2) while loop

જ્યારે condition સાચી હોય ત્યાં સુધી loop ચાલે.

Syntax

```
while(condition)
{
    // statements
}
```

Example

```
int i = 1;
while(i <= 5)
{
    System.out.println(i);
    i++;
}
```

3) do-while loop

આ loop ઓછામાં ઓછો એક વાર તો ચાલે જ છે, પછી condition check થાય છે.

Syntax

```
do
{
    // statements
}
while(condition);
```

Example

```
int i = 1;
do
{
    System.out.println(i);
    i++;
}
while(i <= 5);
```

4) enhanced for loop

Array અથવા collection traverse કરવા માટે.

Example

```
int[] arr = {10, 20, 30, 40};

for(int num : arr)
{
    System.out.println(num);
}
```

Difference between loops

Loop	Use
for	fixed count
while	condition-based
do-while	at least one execution
enhanced for	array/collection traverse

8. 1-D array syntax અને example સાથે સમજાવો

Array શું છે?

Array એ એક એવું data structure છે જેમાં same type ની multiple values એક જ નામ હેઠળ store થાય છે.

1-D Array

One-dimensional array એટલે single row માં values store થવી.

Syntax

```
dataType[] arrayName = new dataType[size];
```

અથવા

```
dataType arrayName[] = new dataType[size];
```

Example

```
class ArrayDemo
{
    public static void main(String[] args)
    {
        int[] marks = {85, 90, 78, 92, 88};

        for(int i = 0; i < marks.length; i++)
        {
            System.out.println("marks[" + i + "] = " + marks[i]);
        }
    }
}
```

Output

```
marks[0] = 85
marks[1] = 90
marks[2] = 78
marks[3] = 92
marks[4] = 88
```

Array ની features

- Same type ની elements store કરે છે
 - Fixed size હોય છે
 - Index 0 થી શરૂ થાય છે
 - Random access શક્ય છે
-

Array નો use કેમ?

જો 100 students ની marks store કરવાની હોય, તો 100 અલગ variables લખવાને બદલે array વધારે useful છે.

Conclusion

Java એક powerful અને widely used programming language છે.

તેની features, data types, operators, loops, switch statement, type casting અને arrays programming માટે ખૂબ important છે.

આ concepts સમજ્યા પછી Java ની basic programming મજબૂત થાય છે.

જો તમારે જોઈએ તો હું આ જ answer ને **exam-style short notes** અથવા **points** માં લખાયેલ **ready-to-write answer** તરીકે પણ આપી શકું છું.